Thực nghiệm giải N-Queens bằng SAT, CP và MIP đến N=1000: báo cáo đầy đủ

Bùi Đình Cảnh[a]

[a]Sinh viên 24021392, lớp INT3011E_3, Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội; email: buidinhcanh2006@gmail.com

ARTICLE INFO

Keywords:
N-Queens
SAT solving
cardinality encoding
PySAT
constraint programming
mixed-integer programming

Abstract

Bài báo nghiên cứu các mô hình giải chính xác cho bài toán N-Queens, với trọng tâm là ảnh hưởng của kỹ thuật mã hóa ràng buộc at-most-one trong SAT. Năm mã hóa Pairwise, Sequential Counter, Commander, Product và Binary được triển khai trên PySAT/Glucose3 và so sánh với OR-Tools CP-SAT, Gurobi MIP, CPLEX MIP và CPLEX CP Optimizer. Quy trình thực nghiệm có timeout cưỡng bức, checkpoint, kiểm tra nghiệm và phân biệt timeout, giới hạn giấy phép và tiến trình bị hệ điều hành dừng. Bộ dữ liệu sau khi bổ sung Gurobi Academic WLS gồm 241 lượt chạy trên các kích thước từ $N = 1$ đến $N = 1000$. Tất cả 182 nghiệm SAT thu được đều vượt qua bộ kiểm tra độc lập; 14 lượt UNSAT tại $N = 2, 3$ cũng phù hợp với tính chất bài toán. Trên môi trường Google Colab đã sử dụng, Gurobi hoàn thành 5/5 lượt tại $N = 64, 128$ và 3/3 lượt tại $N = 256$. Product là mã hóa SAT duy nhất hoàn thành cả ba lần tại $N = 1000$, với trung vị 28.127 giây, 1,298,508 biến và 11,942,164 mệnh đề. Kết quả cũng cho thấy kích thước CNF không đủ để dự đoán thời gian giải: hiệu năng thay đổi không đơn điệu theo N và phụ thuộc mạnh vào cấu trúc encoding cũng như heuristic của solver.

1. Giới thiệu

Bài toán N-Queens yêu cầu đặt N quân hậu trên bàn cờ có N hàng và N cột sao cho không có hai quân nào cùng hàng, cùng cột hoặc cùng đường chéo. Bài toán có mô tả ngắn nhưng là một ví dụ điển hình để khảo sát ảnh hưởng của mô hình hóa tới hiệu năng solver. Cùng một tập ràng buộc có thể được biểu diễn bằng công thức SAT ở dạng CNF, bằng các ràng buộc toàn cục trong Constraint Programming (CP), hoặc bằng hệ ràng buộc tuyến tính trên biến nhị phân trong MIP.

Sự khác biệt giữa ba biểu diễn nằm ở cấu trúc mà solver có thể khai thác. SAT yêu cầu chuyển ràng buộc đếm thành CNF và có thể tạo nhiều biến phụ; CP giữ được ràng buộc `AllDifferent`; MIP dùng biến nhị phân cùng các bất đẳng thức tuyến tính. Bài toán vì thế phù hợp để khảo sát chi phí của mô hình hóa tách biệt với việc tìm một nghiệm khả thi.

Mục tiêu của nghiên cứu không phải đếm mọi lời giải, mà tìm một cấu hình hợp lệ hoặc chứng minh không tồn tại cấu hình. Đóng góp chính gồm: (i) triển khai và kiểm thử năm SAT encoding cho AMO; (ii) xây dựng bốn mô hình đối chứng bằng OR-Tools, Gurobi và CPLEX; (iii) xây dựng pipeline thực nghiệm có thể tiếp tục sau khi Colab bị ngắt; và (iv) đánh giá đồng thời thời gian, quy mô encoding và tỷ lệ hoàn thành đến $N = 1000$.

2. Bối cảnh và câu hỏi nghiên cứu

Các solver SAT hiện đại kết hợp tìm kiếm CDCL và học mệnh đề; hiệu năng của chúng phụ thuộc vào công thức CNF được cung cấp [1]. Đối với ràng buộc at-most-one (AMO), Pairwise ngắn gọn và không dùng biến phụ, trong khi Sequential Counter dùng biến trạng thái để giảm số clause [8]. PySAT cung cấp nhiều kiểu cardinality encoding với cùng giao diện [6, 7]. N-Queens cũng có mô hình CP tự nhiên với ba ràng buộc `AllDifferent` [3].

Báo cáo trả lời ba câu hỏi: (i) kích thước CNF của năm cách mã hóa AMO thay đổi như thế nào theo N; (ii) sự thay đổi đó có dự báo được thời gian giải hay không; và (iii) SAT, CP-SAT và MIP khác nhau ra sao về tỷ lệ hoàn thành trong cùng giới hạn thời gian. So sánh thực nghiệm được giới hạn bởi những giá trị N đã chạy được và điều kiện cấp phép của từng solver.

3. Phạm vi so sánh và nền tảng liên quan

N-Queens là bài toán khả thi: với một N cố định, đầu ra cần là một bàn cờ hợp lệ hoặc chứng nhận không có nghiệm. Cách đặt vấn đề này quan trọng khi so sánh solver. SAT và CP-SAT có thể trả về phép gán; MIP cũng chỉ cần

một nghiệm khả thi, không có giá trị mục tiêu để tối ưu. Do đó thời gian trong báo cáo chỉ nói về việc tìm một nghiệm hoặc kết thúc với kết luận UNSAT trong giới hạn đã cấu hình, không nói về số nghiệm hay chất lượng tối ưu.

Ba họ mô hình có cùng tập nghiệm nhưng trình bày thông tin cho solver theo những cách khác nhau. SAT chuyển các nhóm đếm thành CNF; PySAT cung cấp giao diện tạo ràng buộc cardinality và gọi Glucose3 [6, 7]. CP giữ biến nguyên theo hàng và ba nhóm `AllDifferent`, đúng với mô hình N-Queens của OR-Tools [3]. MIP giữ biến nhị phân theo ô cùng các tổng tuyến tính. Vì vậy số biến của SAT và MIP không so trực tiếp với số biến CP: biến nguyên CP có miền N giá trị, còn các biến Boolean SAT có thể bao gồm biến phụ của encoding.

Đối với SAT, sự khác biệt giữa các mã hóa AMO có hai tầng. Tầng thứ nhất là chi phí dựng công thức: số biến phụ, số clause và độ dài clause. Tầng thứ hai là hiệu quả tìm kiếm sau khi công thức đã được đưa vào Glucose3: mức độ lan truyền, học clause và lựa chọn biến. Sequential Counter giảm kích thước clause theo nhóm so với Pairwise [8], nhưng điều đó không tự nó bảo đảm thời gian giải thấp hơn. Báo cáo vì thế trình bày cả cấu trúc CNF, thời gian và tỷ lệ hoàn thành; một con số thời gian đơn lẻ không đủ.

Phạm vi thực nghiệm là các N xuất hiện trong bộ dữ liệu đã công bố: $1, 2, 3, 4, 8, 16, 32, 64, 128, 256, 512, 1000$. Không phải mọi phương pháp đều được chạy ở mọi N. Những ô không có dữ liệu được giữ là không chạy, không diễn giải là thất bại. Các so sánh có ý nghĩa nhất nằm trong cùng kích thước và cùng giới hạn thời gian; riêng Gurobi ở $N = 64, 128, 256$ được bổ sung ở phiên Colab khác nên cần đọc với lưu ý về môi trường.

4. Mô hình bài toán

Với mỗi ô (r, c), đặt biến Boolean $x_{r,c}$ bằng 1 khi có hậu tại ô đó. Mô hình SAT và MIP sử dụng các nhóm ràng buộc sau:

$$\sum_{c=0}^{N-1} x_{r,c} = 1 \qquad \text{với mọi hàng } r, \tag{1}$$

$$\sum_{r=0}^{N-1} x_{r,c} = 1 \qquad \text{với mọi cột } c, \tag{2}$$

$$\sum_{(r,c) \text{ thuộc } d} x_{r,c} \le 1 \qquad \text{với mọi đường chéo } d. \tag{3}$$

Mỗi ô có đúng một biến cơ sở, nên mô hình SAT/MIP bắt đầu với N^2 biến. Mô hình MIP cài đặt $2N$ ràng buộc hàng/cột và $4N - 2$ ràng buộc đường chéo, trong đó các đường chéo chỉ có một ô tạo bất đẳng thức dư nhưng không làm thay đổi tập nghiệm. SAT chỉ sinh AMO cho đường chéo dài ít nhất hai ô.

Trong SAT, ràng buộc bằng 1 được tách thành at-least-one (ALO) và at-most-one (AMO). Với nhóm literal $X = (x_1, \ldots, x_k)$, ALO là clause $(x_1 \text{ OR } \cdots \text{ OR } x_k)$. AMO được thay đổi theo từng encoding. Trong CP, mỗi hàng i dùng một biến nguyên q_i nhận giá trị từ 0 đến $N - 1$ để biểu diễn cột của quân hậu; mô hình chỉ cần ba ràng buộc toàn cục `AllDifferent` trên q_i, $q_i + i$ và $q_i - i$, đúng với mô hình N-Queens được OR-Tools công bố [3].

Hai mô hình biểu diễn cùng một tập cấu hình: hàng r được gán cột $q_r = c$ đúng khi $x_{r,c} = 1$. `AllDifferent` trên q_r loại xung đột cột; hai biểu thức còn lại lần lượt loại hai hướng đường chéo. Điều này cho phép kiểm tra nghiệm do mọi solver trả về bằng cùng một hàm xác minh độc lập.

4.1. Từ bàn cờ sang ràng buộc và biến CNF

Trong mã nguồn, biến ô (r, c) được đánh số

$$v(r, c) = rN + c + 1, \qquad 0 \le r, c < N. \tag{4}$$

Đánh số từ 1 phù hợp với quy ước literal của PySAT; literal âm biểu thị phủ định. Với $N = 4$, nghiệm có các ô $(0, 1), (1, 3), (2, 0), (3, 2)$ tương ứng các biến $2, 8, 9, 15$. Đây là một ví dụ kiểm tra chuyển đổi chỉ số, không phải nghiệm duy nhất của $N = 4$.

Mỗi hàng và mỗi cột nhận một clause ALO. Chẳng hạn, hàng r có

$$(x_{r,0} \vee x_{r,1} \vee \cdots \vee x_{r,N-1}). \tag{5}$$

Table 1
Bậc kích thước của một nhóm AMO gồm k literal. Các hằng số và trường hợp nhóm rất nhỏ phụ thuộc cài đặt cụ thể.

Mã hóa	Biến phụ	Clause
Pairwise	0	$O(k^2)$
Sequential	$O(k)$	$O(k)$
Binary	$O(\log k)$	$O(k \log k)$
Commander	$O(\sqrt{k})$	$O(k\sqrt{k})$
Product	$O(\sqrt{k})$	$O(k)$

AMO trên cùng nhóm ngăn hai ô cùng đúng; ALO cộng AMO tạo exactly-one. Với đường chéo, chỉ cần AMO. Hai họ đường chéo được nhận biết bởi hằng số $r - c$ và $r + c$. Trong cài đặt SAT, đường chéo dài một ô được bỏ qua vì AMO trên một literal luôn đúng; MIP vẫn tạo bất đẳng thức tương ứng nhưng nó dư.

Với $N \geq 2$, có $2N$ nhóm hàng/cột và $4N - 6$ đường chéo dài ít nhất hai ô cần AMO. Nếu dùng Pairwise cho mọi nhóm, số clause AMO có thể tính trực tiếp:

$$C_{\text{pair}}(N) = 2N\binom{N}{2} + 4\binom{N}{3} + 2\binom{N}{2}. \tag{6}$$

Hạng đầu thuộc hàng/cột. Mỗi hướng chéo có dãy độ dài $1, 2, \ldots, N-1, N, N-1, \ldots, 1$; cộng số cặp trên cả hai hướng cho hai hạng còn lại. Sau khi thêm $2N$ clause ALO, $N = 8$ cho $C_{\text{pair}}(8) + 16 = 744$ clause, đúng bằng số do chương trình ghi nhận. Đây là phép kiểm tra chéo giữa công thức và implementation, đồng thời giải thích vì sao Pairwise tăng bậc ba theo N.

4.2. Đồng nhất nghiệm giữa ba biểu diễn

Đặt $q_r = c$ khi và chỉ khi $x_{r,c} = 1$. Exactly-one trên từng hàng làm ánh xạ này xác định được. Exactly-one trên cột tương đương các q_r đôi một khác nhau. Hai quân ở hàng $r \neq s$ nằm chung đường chéo chính khi $q_r - r = q_s - s$, và chung đường chéo phụ khi $q_r + r = q_s + s$. Vì vậy ba ràng buộc `AllDifferent` của CP, các nhóm AMO/ALO của SAT và các phương trình/bất phương trình MIP diễn tả cùng tập bàn cờ.

Hàm `validate_solution` không cần tin vào trạng thái do solver báo. Nó đòi đúng N tọa độ trong miền, rồi kiểm tra bốn tập khóa: hàng, cột, $r - c$ và $r + c$. Cách kiểm tra độc lập này phát hiện cả nghiệm trùng ô, sai miền và xung đột đường chéo. Nó được gọi sau mọi lần solver báo SAT; run sai sẽ có trạng thái `INVALID_SOLUTION` thay vì được tính là thành công.

5. Các phương pháp giải

5.1. SAT và các mã hóa AMO

PySAT cung cấp giao diện thống nhất cho solver và các encoding cardinality [7, 6]. Glucose3 được dùng làm SAT backend. Pairwise sinh clause $(\overline{x_i}$ OR $\overline{x_j})$ cho mọi cặp. Sequential Counter dùng biến trạng thái phụ tuyến tính theo kích thước nhóm, dựa trên ý tưởng của Sinz [8]. Binary gán mã bit cho vị trí của literal đúng. Commander chia các literal thành nhóm và tạo biến "chỉ huy" cho mỗi nhóm. Product ánh xạ literal lên một lưới phụ hai chiều và áp dụng AMO cho các chỉ báo hàng/cột.

Pairwise, Sequential và Binary gọi trực tiếp `CardEnc` của PySAT; Commander và Product được cài đặt trong project. ALO được áp dụng cho mọi hàng và cột; AMO được áp dụng cho hàng, cột và đường chéo. Bộ kiểm thử truth-table kiểm tra trực tiếp rằng cả năm cài đặt AMO chấp nhận đúng các phép gán có không quá một literal đúng. Ngoài ra, Glucose3 là solver CDCL có heuristic học clause; những đặc trưng này giải thích vì sao số clause không ánh xạ đơn giản thành thời gian chạy [1].

Với nhóm X gồm k literal, Pairwise tạo $\binom{k}{2}$ clause nhị phân và không cần biến phụ. Sequential Counter biểu diễn thông tin tiền tố đã có literal đúng bằng biến phụ. Binary gán cho literal đúng một mã dài $\lceil \log_2 k \rceil$ bit. Bảng 1 tóm tắt bậc kích thước của từng nhóm AMO theo đúng cách cài đặt trong project; ALO bổ sung đúng một clause độ dài k cho mỗi hàng/cột.

Trong Commander, k literal được chia thành các nhóm kích thước xấp xỉ $\sqrt{k}$. Mỗi nhóm có một biến chỉ huy c_g; hai chiều kéo theo giữa c_g và phép OR của nhóm bảo đảm c_g đúng chính xác khi nhóm có literal đúng. AMO Pairwise

được áp dụng bên trong mỗi nhóm và giữa các chỉ huy. Product đặt các literal lên lưới phụ $p \times q$ với $p = \lceil \sqrt{k} \rceil$, $q = \lceil k/p \rceil$. Một literal đúng kéo theo hai biến chỉ báo hàng và cột tương ứng; AMO Pairwise trên hai tập chỉ báo ngăn đồng thời hai literal đúng. Với nhóm tối đa bốn literal, bản cài đặt dùng Pairwise trực tiếp.

5.2. Cấu tạo cụ thể của năm mã hóa

Các mô tả dưới đây xét một nhóm $X = \{x_1, \ldots, x_k\}$ và chỉ mã hóa AMO. ALO của hàng/cột luôn được thêm riêng, không phụ thuộc lựa chọn mã hóa. Đối với Pairwise, mỗi cặp $i < j$ tạo $(\neg x_i \vee \neg x_j)$, nên có đúng $\binom{k}{2}$ clause và không thêm biến. Đây là lựa chọn dễ kiểm chứng nhất, nhưng trên toàn bàn cờ số clause tăng bậc ba theo N như đã tính ở trên.

Sequential Counter dùng biến phụ để biểu diễn trạng thái tiền tố đã có một literal đúng. Những clause chuyển tiếp bảo đảm nếu một literal mới đúng thì trạng thái tiền tố sau nó đúng; một literal mới không thể đúng khi tiền tố trước đó đã có literal đúng. Project gọi `CardEnc.atmost(..., encoding=EncType.seqcounter)` thay vì tự viết chuỗi clause. Cách triển khai này có số biến phụ và clause tuyến tính theo k; số lượng chính xác phụ thuộc quy tắc biên của thư viện.

Binary, tức `EncType.bitwise` trong PySAT, gán cho vị trí i một mã nhị phân có $\lceil \log_2 k \rceil$ bit phụ. Nếu x_i đúng, các bit phải khớp mã của i. Hai literal khác nhau không thể đồng thời đúng vì chúng đòi ít nhất một bit nhận hai giá trị trái nhau. Kết quả là số bit tăng theo $\log k$, trong khi số clause tăng theo $k \log k$. Mã nhị phân không đồng nghĩa với ít clause nhất; nó đánh đổi số biến phụ lấy các liên hệ giữa literal gốc và bit mã.

Commander do project tự cài đặt. Đặt $g = \lceil \sqrt{k} \rceil$, chia nhóm thành $m = \lceil k/g \rceil$ nhóm có kích thước $g_1, \ldots, g_m$. Mỗi nhóm có một biến chỉ huy c_j. Các clause $(\neg x \vee c_j)$ cho từng thành viên và $(\neg c_j \vee \bigvee_{x \in G_j} x)$ buộc c_j đúng đúng khi nhóm có thành viên đúng. Pairwise AMO được dùng cả bên trong nhóm và trên các chỉ huy. Với nhánh áp dụng khi $k > 3$, số biến phụ là m và số clause theo đúng implementation là

$$C_{\text{cmd}}(k) = \sum_{j=1}^{m} \binom{g_j}{2} + \binom{m}{2} + k + m. \tag{7}$$

Ở các nhóm kích thước không quá ba, code chuyển thẳng sang Pairwise.

Product cũng do project tự cài đặt. Đặt $p = \lceil \sqrt{k} \rceil$ và $q = \lceil k/p \rceil$, rồi gắn mỗi literal với một ô khác nhau trong lưới $p \times q$. Nếu literal đúng, hai chỉ báo hàng và cột của ô đó phải đúng. Pairwise AMO trên các chỉ báo hàng và cột khiến không thể có hai literal gốc đúng. Với $k > 4$, có $p + q$ biến phụ và đúng

$$C_{\text{prod}}(k) = 2k + \binom{p}{2} + \binom{q}{2} \tag{8}$$

clause. Với $k \leq 4$, implementation dùng Pairwise trực tiếp.

Các công thức cho thấy bậc tiệm cận chưa quyết định lựa chọn ở nhóm nhỏ. Ví dụ $k = 8$: Pairwise tạo 28 clause; Commander chia nhóm kích thước $3, 3, 2$ và tạo 21 clause cùng ba biến phụ; Product dùng lưới 3×3 và tạo 22 clause cùng sáu biến phụ. Đây là số cho một nhóm AMO, chưa tính clause ALO hay nhiều nhóm chồng lấn của toàn bàn cờ.

5.3. Chi phí ở quy mô toàn bàn cờ

Có $O(N)$ nhóm AMO độ dài tối đa N và tổng độ dài các nhóm là $O(N^2)$. Do đó Pairwise cho $O(N^3)$ clause; Sequential và Product cho $O(N^2)$; Binary cho $O(N^2 \log N)$; Commander cài đặt ở đây cho $O(N^{5/2})$. Đây là bậc trên theo cách dựng công thức, không phải độ phức tạp thời gian tìm kiếm của Glucose3. Ngay cả khi hai encoding có cùng bậc clause, số literal mỗi clause và cấu trúc lan truyền có thể khác nhau. Vì vậy phần thực nghiệm đo cả số biến, số clause, thời gian và run hoàn thành thay vì chọn encoding bằng phân tích tiệm cận đơn thuần.

5.4. Lập luận bảo toàn nghiệm

ALO cùng AMO trên mỗi hàng và cột buộc đúng một hậu tại mỗi hàng/cột. AMO trên hai họ đường chéo loại mọi cặp hậu tấn công nhau. Vì vậy phép gán CNF thỏa mãn luôn cho một bàn cờ hợp lệ. Chiều ngược lại, từ một bàn cờ hợp lệ đặt các biến ô tương ứng bằng đúng và các biến ô còn lại bằng sai; mọi nhóm ALO/AMO đều thỏa. Các biến phụ của Sequential và Binary có thể được gán theo encoding của PySAT. Với Commander, đặt chỉ huy đúng cho nhóm chứa literal đúng. Với Product, đặt đúng chỉ báo hàng/cột chứa literal đúng. Do đó mọi bàn cờ hợp lệ đều mở rộng thành một phép gán CNF thỏa mãn.

Kiểm thử truth-table xét toàn bộ 2^5 phép gán của nhóm năm literal cho cả năm encoding và so sánh kết quả solver với điều kiện nhiều nhất một bit đúng. Kiểm thử đầu-cuối tiếp tục xác nhận SAT cho $N = 1, 4, \ldots, 8$ và UNSAT cho $N = 2, 3$, đồng thời kiểm tra mọi nghiệm bằng hàm độc lập.

5.5. CP-SAT và CP Optimizer

Mô hình OR-Tools CP-SAT và CPLEX CP Optimizer đều dùng N biến nguyên cùng ba ràng buộc `AllDifferent`. CP-SAT là solver CP trên nền SAT cho biến nguyên [2]; CP Optimizer cung cấp các ràng buộc toàn cục cho bài toán tổ hợp [5]. Mỗi solver được giới hạn một worker để giảm sai lệch do mức độ song song khác nhau.

Mô hình CP không cần liệt kê N^2 biến ô. Mỗi $q_r \in \{0, \ldots, N-1\}$ chọn cột cho hàng r; ba ràng buộc `AllDifferent` lần lượt áp dụng cho các dãy (q_r), $(q_r + r)$ và $(q_r - r)$. Cùng một biểu diễn mức cao được đưa cho hai engine CP, song cơ chế lan truyền và tìm kiếm nội bộ khác nhau nên thời gian vẫn có thể khác đáng kể.

5.6. Gurobi MIP và CPLEX MIP

Hai mô hình MIP dùng N^2 biến nhị phân và các phương trình/bất phương trình ở phần trước. Đây là bài toán khả thi, vì vậy không cần hàm mục tiêu. Gurobi và CPLEX được cấu hình một thread, seed theo lần lặp và time limit nội bộ. Gurobi tuân theo mô hình formulate–solve–query của API tối ưu toán học [4]; CPLEX MIP dùng cùng mô hình để tạo phép so sánh giữa hai solver thương mại.

Sau khi solver dừng, nếu có nghiệm khả thi, chương trình lấy các ô có giá trị lớn hơn $0{,}5$ và kiểm tra lại số hậu, miền tọa độ, hàng, cột và đường chéo. Trạng thái hết giờ được phân biệt với vô nghiệm đã chứng minh; lỗi license được ghi thành `LICENSE_LIMIT`, không gán thời gian giải bằng không.

6. Thiết kế thực nghiệm

Thực nghiệm chạy trên Google Colab với 2 CPU logic, một worker cho mỗi solver và bốn phase:

- smoke: $N = 1, 2, 3, 4, 8$, một lần, timeout 15 giây;
- standard: $N = 16, 32, 64, 128$, năm lần, timeout 60 giây;
- CPLEX: $N = 8, 16, 32$, ba lần, timeout 60 giây;
- large: $N = 256, 512, 1000$, ba lần, timeout 120 giây.

Môi trường ghi nhận Linux, Python 3.13.16, PySAT 0.1.8.dev9, OR-Tools 9.15.6755, Gurobi 13.0.3 và CPLEX 22.1.2.1. Pairwise không được chạy ở phase large vì số clause tăng mạnh. Sau đợt đo ban đầu, giấy phép Gurobi Academic WLS đã cho phép chạy lại $N = 64, 128$, năm lần mỗi kích thước, và chạy bổ sung $N = 256$ ba lần. Các file kết quả ban đầu được giữ nguyên; khi gộp, mỗi run Gurobi mới thay thế đúng khóa $(phase, N, method, run)$ cũ bị `LICENSE_LIMIT`. Vì vậy bộ gộp có thêm ba run ở $N = 256$, không nhân đôi mười run tại $N = 64, 128$.

`Total_Time` gồm thời gian xây dựng mô hình/encoding và thời gian giải, nhưng không gồm chi phí khởi tạo process cha. `Wall_Time` bao gồm cả khởi tạo process và IPC. Với Glucose3, seed trong script không được truyền vào solver; vì vậy các lần lặp SAT đo độ dao động thời gian trên cùng cấu hình tìm kiếm mặc định, không phải các lần chạy ngẫu nhiên độc lập.

Mỗi lượt chạy được thực hiện trong process riêng. Process cha đợi tối đa giới hạn thời gian đã chọn, sau đó dừng process con nếu cần; ngay sau mỗi lượt, một dòng CSV được ghi và đồng bộ xuống đĩa để có thể tiếp tục khi Colab ngắt. Chỉ các run có trạng thái SAT hoặc UNSAT mới được dùng để tính trung vị, min và max của `Total_Time`; tỷ lệ thành công được tính trên toàn bộ run, kể cả timeout và lỗi. Không nội suy thời gian cho run chưa hoàn thành.

Các run bổ sung dùng một phiên Colab khác đợt đo gốc. Metadata mới ghi nhận 2 CPU logic và 12,67 GiB RAM, nhưng metadata cũ không lưu model CPU và tổng RAM. Vì vậy các so sánh thời gian giữa Gurobi bổ sung và các solver đã đo trước đó chỉ mang tính mô tả, không phải một phép so sánh phần cứng được kiểm soát chặt.

6.1. Ma trận cấu hình và số lần quan sát

Bảng 2 cho biết chính xác các cấu hình được thực thi. Nhóm cơ sở gồm năm SAT encoding, OR-Tools CP-SAT và Gurobi MIP. Ở phase lớn, Pairwise không được chạy vì quy mô clause dự kiến rất cao, còn MIP Gurobi được bổ sung riêng đến $N = 256$ sau khi có Academic WLS. Hai solver CPLEX có phase riêng vì CP Optimizer cần executable của IBM ngoài gói Python.

Table 2
Các phase trong bộ dữ liệu cuối. Số run là số dòng raw sau gộp.

Phase	Kích thước N	Phương pháp	Lặp	Số run
Smoke	$1, 2, 3, 4, 8$	7	1	35
Standard	$16, 32, 64, 128$	7	5	140
CPLEX	$8, 16, 32$	2	3	18
Large	$256, 512, 1000$	5	3	45
Gurobi WLS bổ sung	256	1	3	3
Tổng				241

Mười run Gurobi tại $N = 64, 128$ trong phase Standard không được tính thêm: chúng thay đúng mười dòng trước đó bị `LICENSE_LIMIT` theo khóa $(phase, N, method, run)$. Ba run Gurobi $N = 256$ là các dòng mới của phase Large. Trong code, `combine_results.py` nối CSV các phase, chuẩn hóa trạng thái legacy và giữ bản ghi cuối với từng khóa. CSV gốc từng phase không bị viết đè; file gộp là sản phẩm phân tích.

6.2. Định nghĩa thời gian và trạng thái

`Encoding_Time` đo thời gian tạo mô hình hoặc CNF trong mỗi worker; `Solving_Time` đo từ lúc gọi engine cho đến lúc có kết quả. Riêng SAT, phần `Solving_Time` còn chứa việc khởi tạo Glucose3 và đưa từng clause vào solver; với MIP nó chủ yếu bao gồm lời gọi `optimize`/`solve`. Vì ranh giới API khác nhau, hai cột này được dùng để hiểu mỗi phương pháp chứ không so trực tiếp như một phân rã hoàn toàn đồng nhất. `Total_Time` bắt đầu trước khi dựng mô hình và kết thúc sau khi solver trả về; `Wall_Time` của process cha còn gồm khởi động process con và chuyển kết quả qua queue.

Process cha áp dụng timeout cưỡng bức. Nếu worker còn chạy khi hết hạn, run được ghi `TIMEOUT` và không có thời gian giải hay số biến/clause hoàn chỉnh. Nếu worker trả lời, trạng thái `SAT` chỉ được giữ khi nghiệm qua kiểm tra độc lập; `UNSAT` là kết luận của solver. Lỗi giới hạn giấy phép được tách thành `LICENSE_LIMIT`. Mã thoát -9 được ghi `KILLED`; không tự đồng nhất với OOM khi không có log hệ thống. Bởi vậy run không hoàn thành không được gán `Total_Time`=0 hay giả sử bằng ngưỡng timeout.

Thống kê thời gian cho một cặp $(N, method)$ là trung vị, min và max trên các run có trạng thái SAT hoặc UNSAT. Tỷ lệ hoàn thành là số run như vậy chia tổng số run đã thực hiện. Với cỡ mẫu chỉ một, ba hoặc năm run, trung vị bền hơn trước một lần chậm bất thường nhưng không phải ước lượng chính xác cho phân bố thời gian tổng quát; báo cáo không tuyên bố ý nghĩa thống kê giữa các solver.

6.3. Môi trường, seed và khả năng tái lập

Mỗi solver được cấu hình một worker. MIP và CP được truyền seed theo chỉ số lần lặp, còn SAT Glucose3 trong cài đặt hiện tại không nhận seed. Do đó lặp lại SAT chủ yếu cho thấy biến thiên runtime trong Colab và đường đi mặc định của solver. Metadata phase gốc ghi Linux, Python 3.13.16, PySAT 0.1.8.dev9, OR-Tools 9.15.6755, Gurobi 13.0.3 và CPLEX 22.1.2.1. Phiên Gurobi Academic WLS bổ sung ghi hai CPU logic, khoảng 12,67 GiB RAM và Gurobi 13.0.3; CPU model chi tiết không được ghi. Metadata gốc không lưu CPU model và RAM, nên không thể hiệu chỉnh khác biệt phần cứng giữa hai đợt đo. Mọi kết luận thứ hạng trong báo cáo là mô tả bộ dữ liệu này, không phải kết quả benchmark kiểm soát máy hoàn toàn.

7. Kết quả và thảo luận

7.1. Tính đúng đắn và mức độ hoàn thành

Bộ dữ liệu có 241 run, không có khóa $(phase, N, method, run)$ bị trùng. Tổng hợp trạng thái được trình bày trong Bảng 3. Tất cả 182 nghiệm SAT đều có `Solution_Valid=True`; 14 kết quả UNSAT đều thuộc $N = 2$ và $N = 3$. Không có `INVALID_SOLUTION`. Ba run `LICENSE_LIMIT` còn lại đều thuộc CPLEX MIP, không thuộc Gurobi.

Hình 1 thể hiện số lần thành công trên tổng số lần chạy. Cần đọc hình này cùng đồ thị thời gian: trung vị chỉ được tính trên các run thành công, nên một điểm thời gian không biểu diễn xác suất hoàn thành.

Table 3
Phân bố trạng thái của toàn bộ lượt chạy.

Trạng thái	Số lượt
SAT	182
UNSAT	14
TIMEOUT	39
LICENSE_LIMIT	3
KILLED	3
Tổng	241

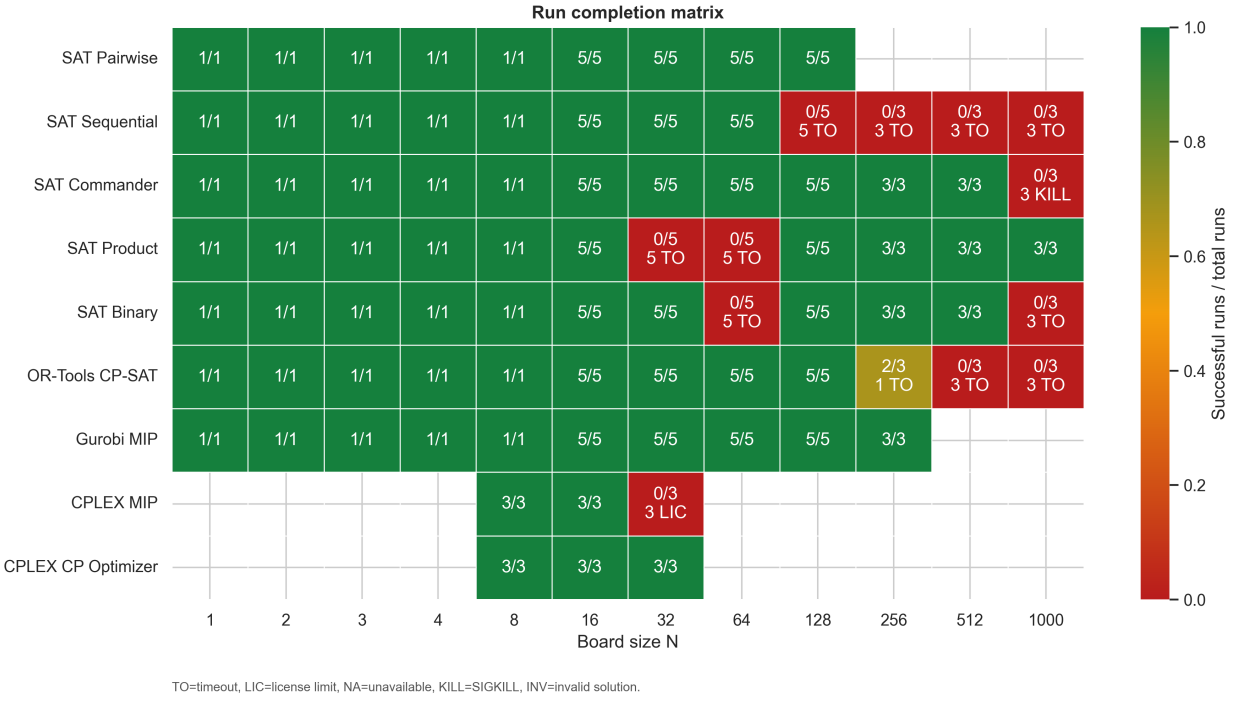


Figure 1: Ma trận hoàn thành. TO: timeout; LIC: giới hạn license; KILL: tiến trình nhận SIGKILL. Ô trống là cấu hình không được đưa vào phase đó.

7.2. Thời gian giải

Hình 2 và Bảng 4 cho thấy không có một phương pháp thắng ở mọi kích thước. Sequential có trung vị thấp nhất tại $N = 16$; Gurobi MIP tại $N = 32, 64, 128$; Product tại $N = 256, 1000$; Binary tại $N = 512$. Với $N = 64, 128$, Gurobi hoàn thành 5/5 run, trung vị lần lượt 0,098 và 0,194 giây. Tại $N = 256$, Gurobi hoàn thành 3/3 run với trung vị 3,078 giây, còn Product là 2,704 giây. Những thứ hạng này mô tả bộ dữ liệu đã gộp, không khẳng định ưu thế tổng quát giữa các solver.

Bảng 5 đặt các run mới cạnh những phương pháp đã chạy trên cùng kích thước. Tại $N = 64$, Product và Binary đều timeout 5/5 lần trong khi Pairwise, CP-SAT và Gurobi hoàn thành. Ở $N = 128$ cả Product và Binary đều trở lại thành công 5/5 lần, nhưng trung vị vẫn lớn hơn Gurobi trong bộ dữ liệu này. Tại $N = 256$, CP-SAT chỉ hoàn thành 2/3 run và Pairwise không được đưa vào phase lớn.

Bảng 6 tập trung vào SAT. Product timeout ở $N = 32, 64$ nhưng hoàn thành cả ba lần ở $N = 1000$; Binary timeout tại $N = 64$ nhưng hoàn thành tại $N = 128, 256, 512$. Đây là bằng chứng thực nghiệm rằng độ khó tìm kiếm không tăng đơn điệu theo N đối với một encoding/heuristic cố định. Không nên diễn giải hiện tượng này thành bảo đảm rằng Product luôn tốt ở kích thước lớn.

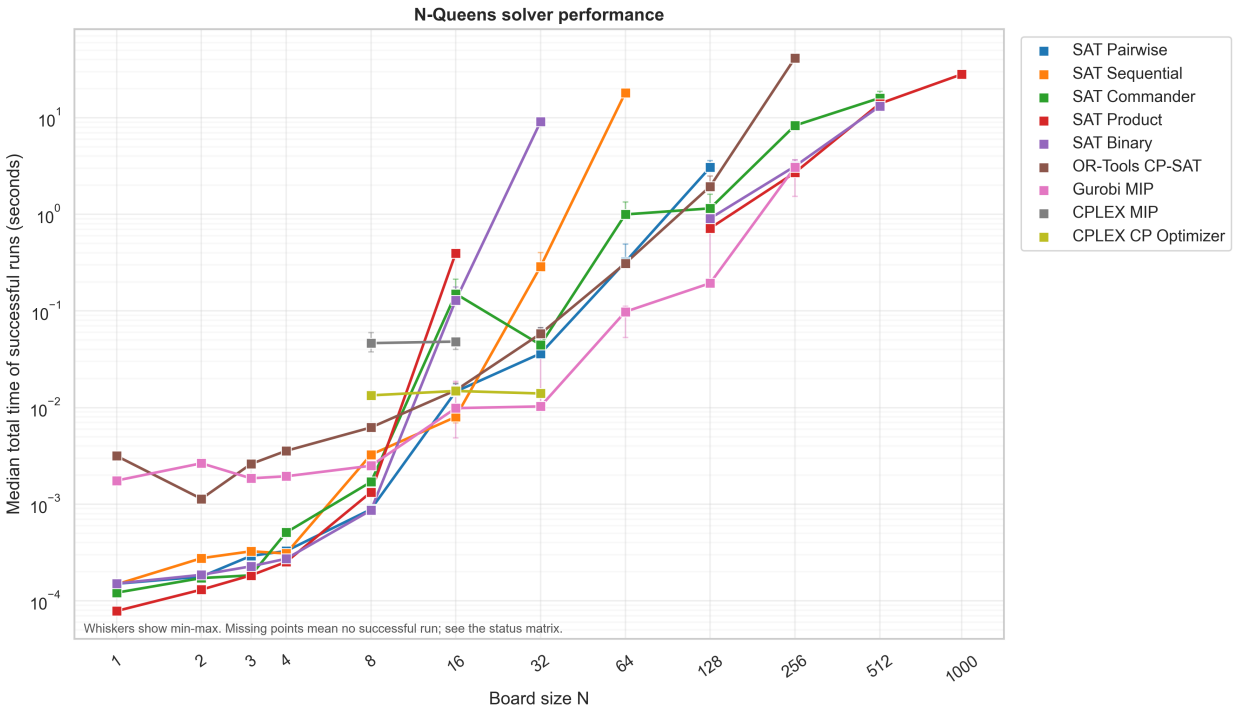


Figure 2: Trung vị thời gian của các run thành công; whisker là min–max. Đường không được nối qua kích thước không có run thành công.

Table 4
Phương pháp có trung vị thấp nhất trong các cấu hình đã chạy.

N	Phương pháp nhanh nhất	Trung vị (s)	Thành công
16	SAT Sequential	0.0080	5/5
32	Gurobi MIP	0.0103	5/5
64	Gurobi MIP	0.0982	5/5
128	Gurobi MIP	0.1939	5/5
256	SAT Product	2.7036	3/3
512	SAT Binary	13.1581	3/3
1000	SAT Product	28.1271	3/3

OR-Tools hoàn thành 2/3 lần ở $N = 256$ với thời gian thành công khoảng 40–43 giây, sau đó timeout hoàn toàn tại $N = 512, 1000$. CPLEX CP hoàn thành 3/3 lần tại $N = 8, 16, 32$; CPLEX MIP gặp giới hạn license ở $N = 32$. Trong dữ liệu gốc, Gurobi bị giới hạn tại $N = 64, 128$; các kết quả đó đã được thay bằng run Academic WLS hợp lệ. Giới hạn cấp phép CPLEX còn lại phản ánh môi trường, không phải thất bại thuật toán.

Ba run Commander tại $N = 1000$ kết thúc với exit code -9, tức SIGKILL. Điều này phù hợp với tình huống process bị hệ thống Colab dừng do áp lực tài nguyên, nhưng không có log kernel để khẳng định chắc chắn nguyên nhân là OOM; do đó dữ liệu được gắn nhãn thận trọng là `KILLED`.

Table 5
Trung vị thời gian theo giây của các run thành công; trong ngoặc là số run thành công/tổng số run. TO: toàn bộ run timeout; dấu gạch ngang: không chạy.

N	Gurobi MIP	CP-SAT	SAT Pairwise	SAT Product	SAT Binary
64	0.098 (5/5)	0.311 (5/5)	0.322 (5/5)	TO	TO
128	0.194 (5/5)	1.944 (5/5)	3.063 (5/5)	0.719 (5/5)	0.908 (5/5)
256	3.078 (3/3)	41.473 (2/3)	–	2.704 (3/3)	3.147 (3/3)

Table 6
Thời gian trung vị SAT theo giây; trong ngoặc là số run thành công/tổng số run. TO: timeout; KILL: SIGKILL; "–": không chạy.

N	Pairwise	Sequential	Commander	Product	Binary
16	0.015 (5/5)	0.008 (5/5)	0.151 (5/5)	0.397 (5/5)	0.129 (5/5)
32	0.036 (5/5)	0.288 (5/5)	0.045 (5/5)	TO	9.090 (5/5)
64	0.322 (5/5)	18.143 (5/5)	0.998 (5/5)	TO	TO
128	3.063 (5/5)	TO	1.153 (5/5)	0.719 (5/5)	0.908 (5/5)
256	–	TO	8.309 (3/3)	2.704 (3/3)	3.147 (3/3)
512	–	TO	16.003 (3/3)	14.004 (3/3)	13.158 (3/3)
1000	–	TO	KILL	28.127 (3/3)	TO

7.3. Quy mô SAT encoding

Hình 3 và Hình 4 cho thấy Pairwise không dùng biến phụ nhưng số clause tăng nhanh. Product tạo nhiều biến hơn nhưng có số clause thấp hơn rõ rệt ở các kích thước lớn quan sát được. Tại $N = 1000$, Product có 1,298,508 biến và 11,942,164 clause, vẫn hoàn thành với trung vị 28.127 giây. Các run timeout/killed không trả lại thống kê encoding, vì vậy đồ thị dừng đường tại các điểm thiếu thay vì nội suy qua chúng.

7.4. Ảnh hưởng của cấu trúc encoding

Tại $N = 64$, Sequential có 47,752 clause và trung vị 18,143 giây; Pairwise có 428,864 clause nhưng trung vị chỉ 0,322 giây. Cả hai hoàn thành 5/5 lần. Đây là một đối chứng rõ ràng chống lại cách chọn encoding chỉ bằng số clause. Ít clause hơn không bảo đảm tìm kiếm nhanh hơn vì công thức với biến phụ tạo cấu trúc lan truyền và lựa chọn nhánh khác. Ở $N = 128$, Product có 192,890 clause với trung vị 0,719 giây, còn Pairwise có 3,462,784 clause và trung vị 3,063 giây. Như vậy quan hệ giữa kích thước CNF và thời gian thay đổi theo cả encoding lẫn kích thước.

Kết quả Product cũng không đơn điệu: toàn bộ năm run ở $N = 32, 64$ đều timeout với ngưỡng 60 giây, nhưng cả năm run ở $N = 128$ hoàn thành và cả ba run ở $N = 1000$ hoàn thành với ngưỡng 120 giây. Giới hạn thời gian ở phase Large dài hơn, nên không thể đặt hai phase này thành một đường cong cùng điều kiện. Ngay trong cùng phase Standard, việc Product timeout ở $N = 64$ rồi thành công ở $N = 128$ cho thấy kích thước bàn cờ không phải biến giải thích duy nhất; heuristic và cấu trúc CNF có thể tương tác phức tạp. Dữ liệu không đủ để nhận diện một cơ chế nhân quả duy nhất.

Ở $N = 1000$, Product tạo 1,298,508 biến và 11,942,164 clause. Thời gian dựng CNF trung vị là 7,736 giây; phần gọi solver, bao gồm nạp clause, có trung vị 20,391 giây; trung vị tổng thời gian là 28,127 giây. Hai thành phần không nên cộng như những trung vị độc lập để suy ra chính xác trung vị tổng, nhưng chúng cho thấy cả chi phí dựng lẫn chi phí solver đều đáng kể ở kích thước này.

7.5. Chi tiết 13 run Gurobi sau khi cấp Academic WLS

Bảng 7 giữ từng lượt Gurobi bổ sung, thay vì chỉ trình bày một trung vị. Tất cả 13 run đều có nghiệm hợp lệ. Tại $N = 128$, bốn run dưới 0,58 giây và một run 0,879 giây; trung vị 0,194 giây thể hiện nhóm thời gian thấp nhưng không

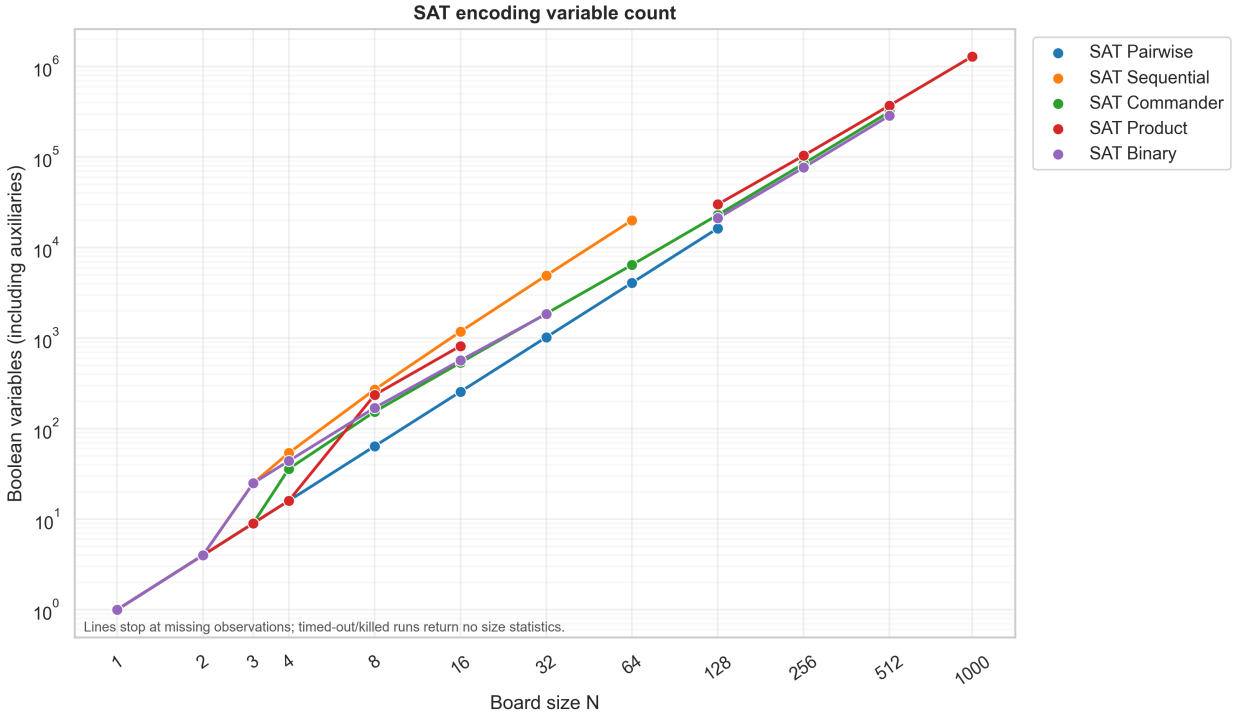


Figure 3: Số biến Boolean, bao gồm biến phụ, của các SAT encoding.

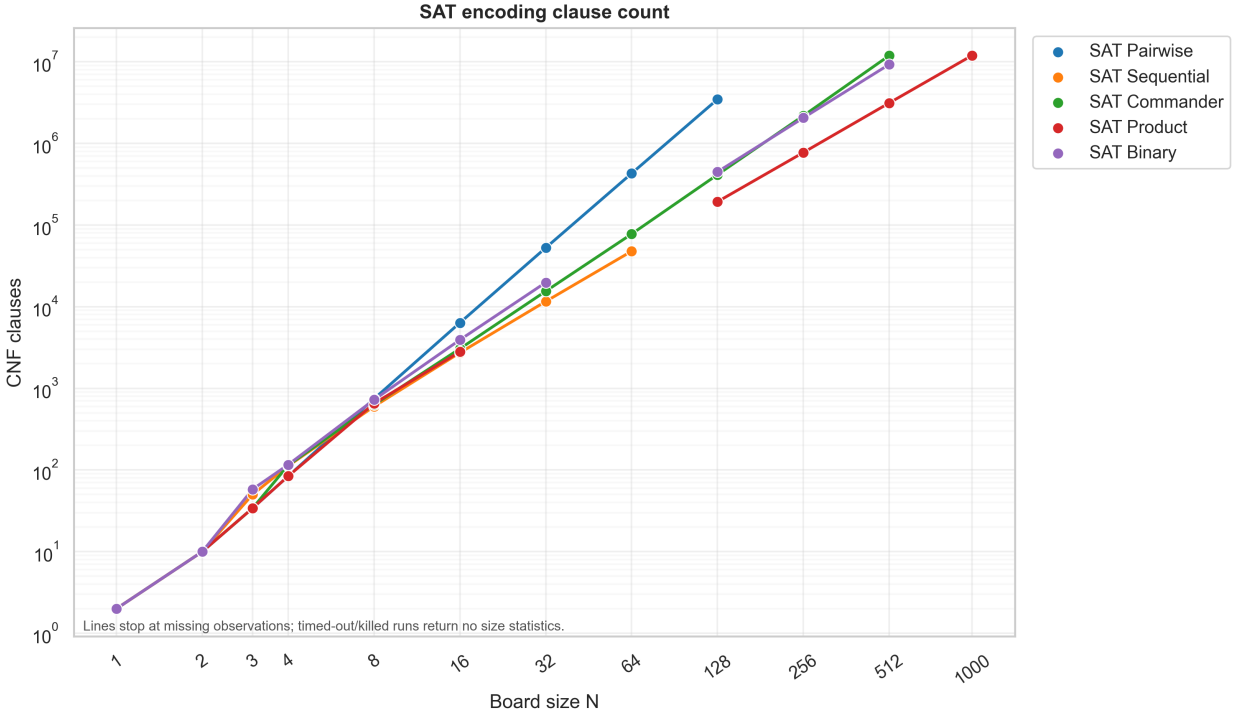


Figure 4: Số clause CNF của các SAT encoding.

Table 7
Thời gian từng run Gurobi Academic WLS tại các kích thước bổ sung. Các run đều có trạng thái SAT và nghiệm hợp lệ.

N	Lượt	Dựng mô hình (s)	Giải (s)	Tổng (s)
64	1	0.1044	0.0070	0.1114
64	2	0.0946	0.0036	0.0982
64	3	0.0966	0.0035	0.1001
64	4	0.0557	0.0020	0.0577
64	5	0.0502	0.0030	0.0532
128	1	0.1866	0.0073	0.1939
128	2	0.1835	0.0077	0.1912
128	3	0.1853	0.6941	0.8794
128	4	0.1837	0.0069	0.1906
128	5	0.1826	0.3969	0.5795
256	1	0.9886	2.6657	3.6543
256	2	0.7813	2.2962	3.0775
256	3	1.4802	0.0592	1.5393

Table 8
Kết quả hai solver IBM trên Colab. Dấu – là không có trung vị thời gian của run hoàn thành.

Bộ giải	N	Hoàn thành	Trung vị (s)	Trạng thái
CPLEX MIP	8	3/3	0.0465	SAT
CPLEX MIP	16	3/3	0.0482	SAT
CPLEX MIP	32	0/3	–	LICENSE_LIMIT
CPLEX CP	8	3/3	0.0134	SAT
CPLEX CP	16	3/3	0.0149	SAT
CPLEX CP	32	3/3	0.0140	SAT

che giấu biến thiên giữa các lần đo. Tại $N = 256$, khoảng thời gian là 1,539–3,654 giây. Sự biến thiên này là một lý do không dựa vào một run đơn lẻ để kết luận ưu thế của solver.

Mô hình Gurobi tại $N = 64, 128, 256$ lần lượt có $4{,}096$, $16{,}384$ và $65{,}536$ biến nhị phân, cùng 382, 766 và $1{,}534$ ràng buộc. Những số ràng buộc khớp $6N - 2$ từ mô hình đã trình bày. So sánh thời gian Gurobi với SAT/CP trong cùng bảng chỉ mang tính tham chiếu: các run Academic WLS được tạo ở phiên Colab khác, dù cùng pipeline và giới hạn thời gian tương ứng.

7.6. Hai cách dùng CPLEX trong thực nghiệm

Bảng 8 tách CPLEX MIP và CPLEX CP Optimizer. Đây là hai solver khác mô hình và khác hệ cấp phép, không phải hai tên cho cùng một run. CP Optimizer giải thành công 3/3 lần ở $N = 8, 16, 32$ với mô hình N biến nguyên và ba `AllDifferent`. CPLEX MIP giải thành công 3/3 lần ở $N = 8, 16$, nhưng cả ba run $N = 32$ gặp `LICENSE_LIMIT` trước khi có thống kê thời gian giải. Gurobi Academic WLS không mở khóa license CPLEX MIP.

Hai unit test CPLEX có thể bị skip khi chạy trên máy local nếu thiếu thư viện `cplex`/`docplex` hoặc executable `cpoptimizer`. Việc skip đó phản ánh dependency trên máy local, không phủ nhận 18 run CPLEX trong dữ liệu Colab: 15 SAT hợp lệ và ba `LICENSE_LIMIT`. Khi trình bày kết quả, cần giữ ranh giới giữa trạng thái kiểm thử local và kết quả thực nghiệm đã lưu.

8. Hạn chế và nguy cơ ảnh hưởng tính hợp lệ

Thứ nhất, Colab là môi trường chia sẻ; mô hình CPU và tổng RAM chưa được lưu trong bộ metadata cũ, còn Gurobi WLS chạy trong một phiên sau đó. Không nên diễn giải thứ hạng thời gian như một benchmark được kiểm soát phần cứng. Thứ hai, số lần lặp khác nhau giữa các phase (1, 3 hoặc 5), và SAT không dùng seed ngẫu nhiên. Thứ ba, timeout ở process cha làm mất số biến/clause và không cho biết run dừng trong encoding hay solving. Thứ tư, giới hạn license

CPLEX MIP và RAM Colab làm thu hẹp miền kích thước có thể so sánh; Gurobi chỉ được chạy bổ sung đến $N = 256$. Cuối cùng, nghiên cứu chỉ tìm một lời giải khả thi; kết quả không đại diện cho bài toán đếm toàn bộ nghiệm N-Queens.

9. Kết luận

Project đã triển khai đầy đủ các nhóm phương pháp được yêu cầu và xác minh tính đúng đắn trên các trường hợp SAT/UNSAT nhỏ. Bộ dữ liệu 241 run đến $N = 1000$ cho thấy việc chọn encoding có ảnh hưởng quyết định: Pairwise có ít biến nhưng bùng nổ clause, Sequential suy giảm mạnh từ $N = 64$, trong khi Product và Binary mở rộng tốt hơn ở một số kích thước lớn. Gurobi Academic WLS giải thành công toàn bộ run bổ sung tại $N = 64, 128, 256$; song do thực nghiệm qua nhiều phiên Colab, không thể suy rộng thứ hạng thời gian thành ưu thế thuật toán tuyệt đối. Các bất thường không đơn điệu nhấn mạnh rằng không thể đánh giá encoding chỉ bằng số biến hoặc số clause; cần báo cáo đồng thời thời gian, tỷ lệ hoàn thành và giới hạn môi trường.

Data availability

Mã nguồn, unit test, hướng dẫn Colab, dữ liệu thô, bảng tổng hợp và biểu đồ được lưu tại `https://github.com/WilliamsBui-fnd/INT3011E_3`. File `benchmark_raw.csv` được xem là dữ liệu gốc; thư mục `combined_licensed` chứa bản gộp dùng trong báo cáo này, còn các thư mục phase giữ nguyên dữ liệu trước khi thay thế run bị giới hạn license. Mọi bảng và hình trong báo cáo có thể tái tạo bằng `combine_results.py`, `plot_results.py` và `prepare_report.py`.

CRediT authorship contribution statement

Bùi Đình Cảnh: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft, Writing – review and editing.

Declaration of competing interest

Tác giả tuyên bố không có lợi ích tài chính hoặc quan hệ cá nhân có thể ảnh hưởng tới công trình này.

Tuyên bố về công cụ AI hỗ trợ

Trong quá trình chuẩn bị project, tác giả sử dụng ChatGPT/Codex để hỗ trợ rà soát mã nguồn, tổ chức thí nghiệm, trực quan hóa và hiệu đính ngôn ngữ. Tác giả đã kiểm tra mã, nghiệm, số liệu và chịu trách nhiệm cuối cùng về nội dung báo cáo.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence, pp. 399–404.

[2] Google, a. Cp-sat solver. `https://developers.google.com/optimization/cp/cp_solver`. OR-Tools documentation, truy cập ngày 11-10-2026.

[3] Google, b. The n-queens problem. `https://developers.google.com/optimization/cp/queens`. OR-Tools documentation, truy cập ngày 11-10-2026.

[4] Gurobi Optimization, LLC, 2026. Gurobi optimizer reference manual, version 13.0. `https://docs.gurobi.com/projects/optimizer/en/current/`.

[5] IBM, . Ibm ilog cp optimizer overview. `https://www.ibm.com/docs/en/icos/22.1.0?topic=cpco-overview`. Truy cập ngày 11-10-2026.

[6] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. Pysat: A python toolkit for prototyping with sat oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437.

[7] PySAT developers, . Pysat documentation. `https://pysathq.github.io/docs/html/`. Truy cập ngày 11-10-2026.

[8] Sinz, C., 2005. Towards an optimal cnf encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. doi:`10.1007/11564751_73`.

A. Bảng kết quả SAT chi tiết

Bảng 9 liệt kê mọi cấu hình SAT từ $N = 16$ trở lên có trong dữ liệu, kể cả trường hợp không hoàn thành. Cột thời gian ghi trung vị của các run SAT/UNSAT theo giây và số run hoàn thành/tổng số run. TO là toàn bộ run timeout; KILL là toàn bộ run bị SIGKILL. Số biến và clause chỉ có khi worker hoàn tất và trả thống kê; không điền số giả cho run bị dừng. Pairwise không có dòng ở phase Large vì không được đưa vào kế hoạch chạy trên Colab.

Table 9: Quy mô CNF và trung vị thời gian của từng cấu hình SAT đã chạy. Dấu – biểu thị không có thống kê do không chạy hoặc run không hoàn thành.

N	Mã hóa	Biến	Clause	Thời gian/Trạng thái
16	Pairwise	256	6,352	0.015 (5/5)
16	Sequential	1,182	2,728	0.008 (5/5)
16	Commander	536	3,056	0.151 (5/5)
16	Product	820	2,804	0.397 (5/5)
16	Binary	568	3,924	0.129 (5/5)
32	Pairwise	1,024	52,640	0.036 (5/5)
32	Sequential	4,926	11,592	0.288 (5/5)
32	Commander	1,868	15,466	0.045 (5/5)
32	Product	–	–	TO
32	Binary	1,846	19,796	9.090 (5/5)
64	Pairwise	4,096	428,864	0.322 (5/5)
64	Sequential	20,094	47,752	18.143 (5/5)
64	Commander	6,464	77,604	0.998 (5/5)
64	Product	–	–	TO
64	Binary	–	–	TO
128	Pairwise	16,384	3,462,784	3.063 (5/5)
128	Sequential	–	–	TO
128	Commander	23,034	413,632	1.153 (5/5)
128	Product	30,166	192,890	0.719 (5/5)
128	Binary	21,234	447,828	0.908 (5/5)
256	Sequential	–	–	TO
256	Commander	84,624	2,177,772	8.309 (3/3)
256	Product	104,172	772,740	2.704 (3/3)
256	Binary	76,784	2,053,460	3.147 (3/3)
512	Sequential	–	–	TO
512	Commander	316,546	11,889,662	16.003 (3/3)
512	Product	371,940	3,124,284	14.004 (3/3)
512	Binary	287,726	9,262,420	13.158 (3/3)
1000	Sequential	–	–	TO
1000	Commander	–	–	KILL
1000	Product	1,298,508	11,942,164	28.127 (3/3)
1000	Binary	–	–	TO

B. Đối chiếu trạng thái theo phase

Bảng 10: Số run theo trạng thái và phase sau khi gộp Gurobi WLS. Không có run `INVALID_SOLUTION` hoặc `UNAVAILABLE`.

Phase	SAT	UNSAT	TIMEOUT	LICENSE_LIMIT	KILLED	Tổng
Smoke	21	14	0	0	0	35
Standard	120	0	20	0	0	140
CPLEX	15	0	0	3	0	18
Large	26	0	19	0	3	48
Tổng	182	14	39	3	3	241

Phân rã này xác nhận ba điều thường dễ nhầm lẫn: 14 kết quả UNSAT đều từ phase Smoke tại $N = 2, 3$; ba giới hạn license cuối cùng thuộc phase CPLEX; và ba SIGKILL nằm ở phase Large. Không có dòng Gurobi `LICENSE_LIMIT` sau khi thay thế bằng các run Academic WLS.

C. Tái tạo bảng, hình và PDF

Repository tại `https://github.com/WilliamsBui-fnd/INT3011E_3` chứa code, unit test và hai file CSV chuẩn trong `NQueens_BTL1_results/combined_licensed`. CSV raw là nguồn gốc cho mọi tổng hợp; file summary chỉ là kết quả nhóm theo $(N, method)$. Dữ liệu phase ban đầu vẫn được giữ để kiểm tra lịch sử, nhưng gói kết quả đưa về máy có thể chỉ có CSV gộp và metadata của các lượt Gurobi mới. Vì vậy có thể tái tạo báo cáo trực tiếp từ CSV gộp mà không cần license solver:

```
python3 -m unittest discover -s tests -v
python3 plot_results.py \
  --input NQueens_BTL1_results/combined_licensed/benchmark_results.csv \
  --raw NQueens_BTL1_results/combined_licensed/benchmark_raw.csv \
  --output-dir NQueens_BTL1_results/combined_licensed
python3 prepare_report.py
```

Sau đó biên dịch `full_report_template.tex` bằng XeLaTeX (chạy BibTeX và biên dịch lại để cập nhật tài liệu tham khảo). File `report_template.tex` tạo bản ngắn từ cùng nguồn `report/main.tex`. Các bảng ở phụ lục được sinh bởi `prepare_report.py`; không cần sửa số liệu bằng tay. File license Gurobi không được đưa vào Git, CSV hay PDF. Nếu chạy lại solver trên Colab, cần cấp license qua đường dẫn riêng tư theo hướng dẫn `COLAB_GUIDE.md`; việc chỉ tái tạo biểu đồ và báo cáo thì không cần license.

Để kiểm tra dữ liệu mới trước khi thay báo cáo, có thể đối chiếu số run, số khóa $(phase, N, method, run)$ duy nhất, tổng năm trạng thái trong Bảng 10, và cờ `Solution_Valid=True` ở toàn bộ 182 dòng SAT. Việc giữ cột `Error_Type`, `Error_Message` và `Exit_Code` trong CSV raw giúp kiểm toán timeout và SIGKILL mà không diễn giải thiếu bằng chứng thành lỗi thuật toán.

Lược đồ dữ liệu dùng để kiểm toán

Mỗi dòng của `benchmark_raw.csv` là một lượt chạy, không phải một giá trị trung bình. Bốn trường `Phase, N, Method, Run` tạo khóa duy nhất; `Status` và `Solution_Valid` ghi kết luận và kiểm tra nghiệm. Các cột `Variables, Clauses, Constraints` chỉ có khi worker trả thống kê. `Total_Time` của run hoàn thành được dùng tính trung vị; `Wall_Time` đo thêm chi phí process cha. Cuối cùng, `Error_Type`, `Error_Message` và `Exit_Code` cho phép kiểm toán timeout, license limit và SIGKILL.